(GUI) for a student registration form using the Swing framework.

The following Java program creates a graphical user interface (GUI) for a student registration form using the Swing framework. It includes fields for student name, roll number, gender selection, branch, and terms acceptance. The program validates user input and displays a success message upon successful registration. This document will detail each component and line of code to enhance understanding of Java Swing applications.

Code Explanation

```
import javax.swing.*;
import java.awt.event.*;
```

- **Imports**: The program imports necessary classes from the javax.swing package for creating the GUI components and the java.awt.event package for handling events like button clicks.

```
public class StudentRegistration extends JFrame implements ActionListener {
```

- **Class Declaration**: The class StudentRegistration extends JFrame, making it a window-based application. It implements ActionListener to handle button click events.

Instance Variables

```
    // Labels
    JLabel lblName, lblRoll, lblGender, lblBranch;
```

- **Labels**: These variables will hold the labels for the input fields, making it clear what information is required from the user.

```
    // Text Fields
    JTextField txtName, txtRoll, txtBranch;
```

- **Text Fields**: These variables represent the input fields where users will enter their name, roll number, and branch.

```
    // Radio Buttons
    JRadioButton male, female;
    ButtonGroup genderGroup;
```

- **Radio Buttons**: male and female are radio buttons for gender selection. ButtonGroup is used to group these buttons so that only one can be selected at a time.

```
    // Check Box
    JCheckBox terms;
```

- **Check Box**: This variable represents a checkbox for users to accept the terms and conditions.

```
    // Buttons
    JButton submit, reset;
```

- **Buttons**: These variables represent the submit and reset buttons for the form.

Constructor

```
    public StudentRegistration() {
```

- **Constructor**: This method initializes the GUI components and sets up the frame.

```
        setTitle("Student Registration Form");
        setSize(450, 450);
        setLayout(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

- **Frame Properties**: The title of the window is set, its size is defined, the layout is set to null (absolute positioning), and the default close operation is specified.

Adding Components

```
        // Student Name
        lblName = new JLabel("Student Name");
        lblName.setBounds(30, 30, 120, 30);
        add(lblName);
```

- **Label for Name**: A label is created for the student name, positioned at (30, 30) with a width of 120 and height of 30 pixels. It is then added to the frame.

```
        txtName = new JTextField();
        txtName.setBounds(170, 30, 180, 30);
        add(txtName);
```

- **Text Field for Name**: A text field is created for user input, positioned next to the name label, and added to the frame.

```
        // Roll Number
        lblRoll = new JLabel("Roll Number");
        lblRoll.setBounds(30, 80, 120, 30);
        add(lblRoll);
```

- **Label for Roll Number**: Similar to the name label, this label is created for the roll number.

```
        txtRoll = new JTextField();
        txtRoll.setBounds(170, 80, 180, 30);
        add(txtRoll);
```

- **Text Field for Roll Number**: A text field for roll number input is created and added to the frame.

```
        // Gender
        lblGender = new JLabel("Gender");
        lblGender.setBounds(30, 130, 120, 30);
        add(lblGender);
```

- **Label for Gender**: A label for gender selection is created.

```
        male = new JRadioButton("Male");
        male.setBounds(170, 130, 80, 30);
        add(male);
```

- **Radio Button for Male**: A radio button for selecting male is created and positioned.

```
        female = new JRadioButton("Female");
        female.setBounds(260, 130, 100, 30);
        add(female);
```

- **Radio Button for Female**: A radio button for selecting female is created and positioned next to the male button.

```
        genderGroup = new ButtonGroup();
        genderGroup.add(male);
        genderGroup.add(female);
```

- **Button Group**: A button group is created to ensure that only one gender option can be selected at a time.

```
        // Branch
        lblBranch = new JLabel("Branch");
        lblBranch.setBounds(30, 180, 120, 30);
        add(lblBranch);
```

- **Label for Branch**: A label for the branch input is created.

```
        txtBranch = new JTextFieldO();
        txtBranch.setBounds(170, 180, 180, 30);
        add(txtBranch);
```

- **Text Field for Branch**: A text field for branch input is created and added to the frame.

```
        // Terms & Conditions
        terms = new JCheckBox("I accept Terms & Conditions");
        terms.setBounds(30, 230, 250, 30);
        add(terms);
```

- **Checkbox for Terms**: A checkbox for accepting terms and conditions is created and added to the frame.

```
        // Submit Button
        submit = new JButton("Submit");
        submit.setBounds(80, 290, 100, 35);
        submit.addActionListener(this);
        add(submit);
```

- **Submit Button**: A button for submitting the form is created, positioned, and an action listener is added to handle clicks.

```
        // Reset Button
        reset = new JButton("Reset");
        reset.setBounds(220, 290, 100, 35);
        reset.addActionListener(this);
        add(reset);
```

- **Reset Button**: A button for resetting the form is created, positioned, and an action listener is added.

```
        setVisible(true);
    }
```

- **Visibility**: The frame is made visible to the user.

Action Handling

```
    @Override
    public void actionPerformed(ActionEvent e) {
```

- **Event Handling**: This method is overridden to handle button click events.

```
        // Reset Button
        if (e.getSource() == reset) {
```

- **Reset Button Logic**: Checks if the reset button was clicked.

```
            txtName.setText("");
            txtRoll.setText("");
            txtBranch.setText("");
```

- **Clearing Fields**: The text fields are cleared.

```
            genderGroup.clearSelection();
            terms.setSelected(false);
```

- **Clearing Selections**: The gender selection is cleared and the terms checkbox is deselected.

```
            return;
        }
```

- **Exit Reset Logic**: Exits the method after resetting.

```
        // Submit Button
        if (txtName.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this,
                    "Please enter Student Name");
            return;
        }
```

- **Validation for Name**: Checks if the name field is empty and shows a message if it is.

```
        if (txtRoll.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this,
                    "Please enter Roll Number");
            return;
        }
```

- **Validation for Roll Number**: Checks if the roll number field is empty.

```
        if (!male.isSelected() && !female.isSelected()) {
            JOptionPane.showMessageDialog(this,
                    "Please select Gender");
            return;
        }
```

- **Validation for Gender**: Checks if no gender is selected.

```
        if (txtBranch.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this,
                    "Please enter Branch");
            return;
        }
```

- **Validation for Branch**: Checks if the branch field is empty.

```
        if (!terms.isSelected()) {
            JOptionPane.showMessageDialog(this,
                    "Please accept Terms & Conditions");
            return;
        }
```

- **Validation for Terms**: Checks if the terms checkbox is not selected.

```
        String gender = male.isSelected() ? "Male" : "Female";
```

- **Gender Assignment**: Assigns the selected gender to a string variable.

```
        JOptionPane.showMessageDialog(this,
                "Registration Successful\n\n"
                    + "Name : " + txtName.getText()
                    + "\nRoll Number : " + txtRoll.getText()
                    + "\nGender : " + gender
                    + "\nBranch : " + txtBranch.getText());
    }
```

- **Success Message**: Displays a dialog with the registration details upon successful validation.

Main Method

```
    public static void main(String[] args) {
        new StudentRegistration();
    }
}
```

- **Main Method**: The entry point of the program, which creates an instance of the StudentRegistration class, thereby launching the application.

Conclusion

This Java Swing application provides a simple yet effective way to collect student registration information. Each component is carefully designed to ensure user-friendly interaction, and the validation checks enhance data integrity. Understanding this code can serve as a foundation for developing more complex GUI applications in Java.